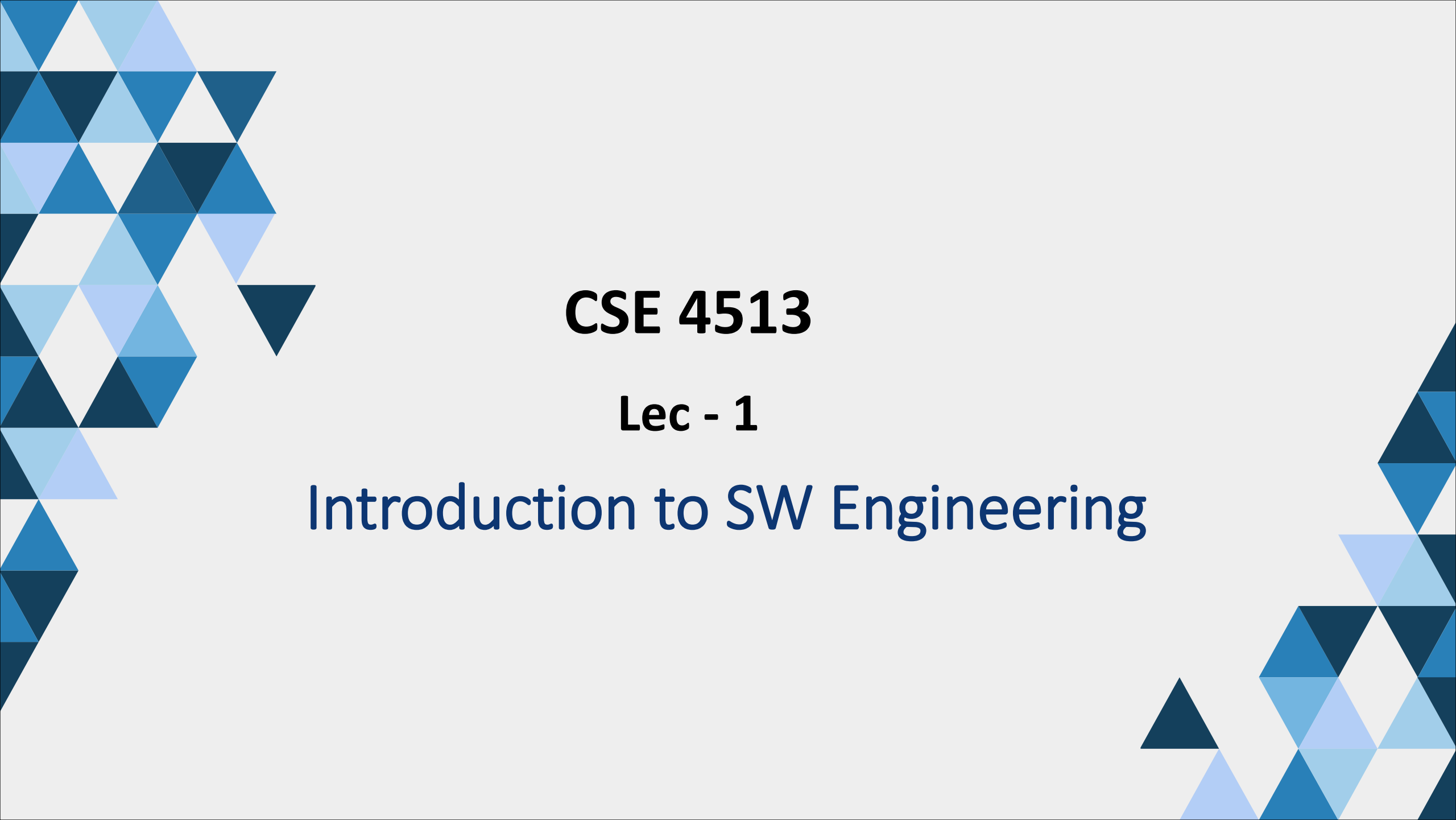


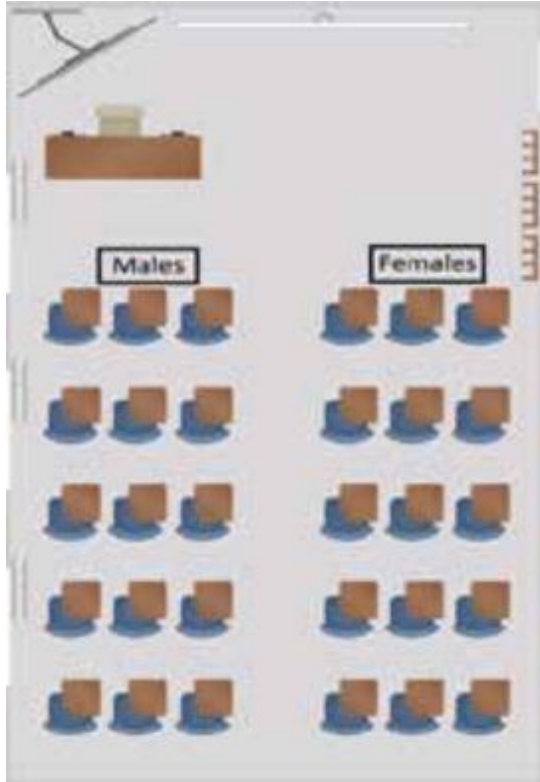


CSE 4513

Lec - 1

Introduction to SW Engineering

CLASSROOM RULES



- Male and female students will sit in different columns/rows in the classroom. No male and female students can seat side-by-side.
- In case of individual sitting arrangement parted (tablet-chair), they may seat “next” to one another.
- If the number of female students is less than that of the male students, female students may seat in the front rows or front desks of the column. The rest of the seats located in the backside could be filled by the male students

✓ **BE ON TIME** (If you're early, you're on time. If you're on time, you're late)

✓ **BE A GOOD LISTENER**

✓ **ASK QUESTIONS**

✓ **SHARE NEW IDEAS**

✓ **CONTRIBUTE TO DISCUSSIONS**

✓ **NO UNETHICAL REQUEST**

CLASS SCHEDULE



	Sec 1	Sec 2
Wednesday	11:45 - 13:00 (304)	9:15 - 10:30 (301)
Friday	15:45 - 17:00(302)	11:45 - 13:00 (302)

GOOGLE CLASSROOM INFO



Join using Class Code

4zyv4le

Join using Invitation link

<https://classroom.google.com/c/NzA1MzUyOTc2NTQ4?cjc=4zyv4le>

Use only IUT email to join

IMPORTANT FOR YOU

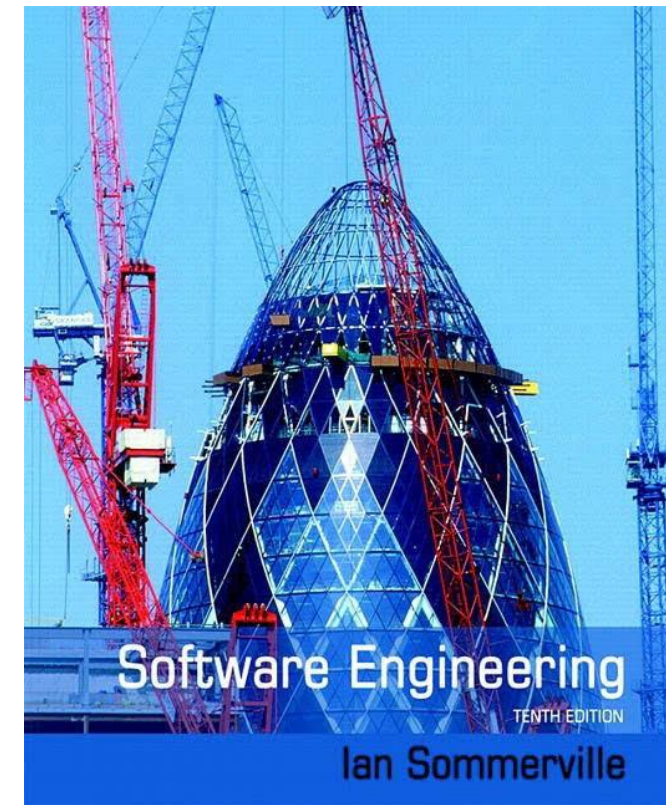


Course Code: CSE 4513

Course Name: Software Engineering and Object-Oriented Design

Textbook:

1. Software Engineering a practitioner's approach, Author: Roger S. Pressman Fourth edition, McGraw-Hill Book Company
2. Object Oriented Modeling and Design, Author: Raumbugh
3. Unified Modeling system, Author: Raumbugh
4. Applying UML and Patterns, Author: Craig larman
5. Object oriented analysis and design Using UML Author: Bennet Farmer
6. Design Pattern, Author: Gamma, Helm & Jhonson



ASSESSMENT & GRADING POLICY



Course Assessment Method

**Attendance
(10%)**

Quiz 15% of Total Marks (Best 3 out of 4)

**Mid
Semester
(25%)**

**Semester
Final (50%)**

Grading Policy

Marks out of 100	Letter Grade	Grade Point	Marks out of 100	Letter Grade	Grade Point
80 - 100	A+	4.00	55 - 59	B-	2.75
75 - 79	A	3.75	50 - 54	C+	2.50
70 - 74	A-	3.50	45 - 49	C	2.25
65 - 69	B+	3.25	40 - 44	D	2.00
60 - 64	B	3.00	00 - 39	F	0.00

COURSE OBJECTIVE



Upon completion of this course, you will be familiar with:

- 01** the process of requirement analysis, designing, implementing and maintaining of Software systems.
- 02** The concept of Object-Orientation, Design patterns and its importance
- 03** the testing approaches and quality assurance to ensure high standard/professional software.

COURSE OUTCOME(CO)



- CO1** Understand the fundamentals of software development process.
- CO2** Asses the SW requirements and applying appropriate design techniques to build SW systems.
- CO3** Demonstrate an ability to use the techniques and tools for to release and maintain quality software.

OUTCOME BASED EDUCATION (OBE)



Exam Result is Not the Most Important Consideration by Employer

Communication (verbal & written)

Honesty/Integrity

Teamwork skills

Interpersonal skills

Strong work ethics

Motivation & initiative

Flexibility/adaptability

Analytical skills

Computer skills

Creativity

Organizational skills

Detail oriented

Leadership skills

Self confidence

Friendly/outgoing personality

Well mannered / polite

Tactfulness

CGPA (3.0 or better)

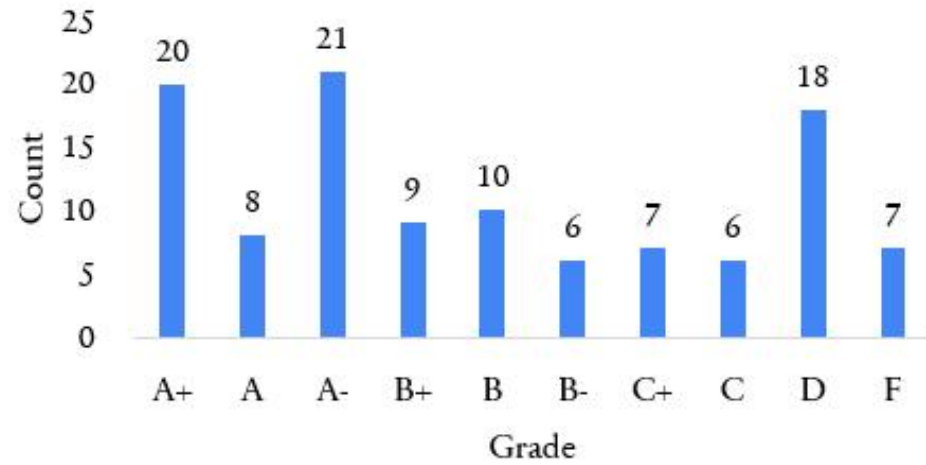
Sense of humor

Entrepreneurial skills/risk taker

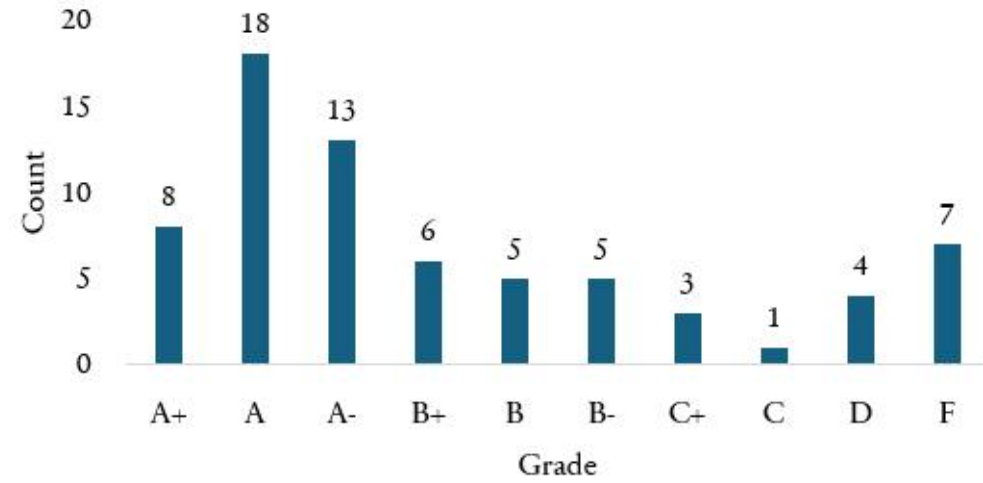
HOW WAS THE EVALUATION



Batch 17



Batch 20



LECTURE PLAN



Weeks	Topics
1	Introduction to SW Engineering, SW size estimation, Cost, Schedule & Quality
2	SW Processes, SW Development Models
3	SW Requirement Analysis
4	UI Specification and UI Design
5	Object modeling Using UML
6	SW Design, OO Basics, SOLID Principle
7	SOLID Principle
8	Design Pattern
9	Design Pattern
10	Clean Coding, Debugging and Unit testing
11	Test Driven development, SW testing
12	SW Testing
13	SW configuration Management, Open-Source Compliance
14	SW release and Maintenance
15	Review Class

LOOKING FOR SW ENGINEER



Software Engineer Experience: At least 1 year

Education

- Bachelor of Computer Science & Engineering.

Job Description:

We are seeking an experienced, self-motivated Senior Software Engineer with 1/2 years of experience developing applications. As a Senior Software Engineer, you will collaboratively identify, design, develop and deploy technology and process solutions to implement highly scalable business applications in Linux and Windows environments. This role requires technical expertise, Banking and NBF domain proficiency and .NET platform hands-on experience.

Responsibilities:

- Collaborates with the development team and initiates process improvements for new and existing systems.
- Participates in the solution design process: requirements gathering, and backlog refinement, shapes, and guides systems approach, helps with project initiation, contributes to functional design, technical design and development.
- Develop software solutions to business problems. Leverages established design patterns and coding standards. Builds solutions that are secure, durable, scalable, flexible, and accessible and proactively mitigate against production support issues.
- Automates test coverage per platform capabilities and requirements. Establishes and maintains continuous build and integration testing on applicable platforms and assists with manual system and integration testing efforts.

LOOKING FOR SW ENGINEER



Software Engineer *Experience: At least 1 year*

Core Competencies:

- **Customer Focus** - Delivers legendary service that meets and exceeds all customers' expectations.
- **Ethics and Integrity** - Adheres to Leads values, beliefs, and principles during good and bad times.
- **Composure** - Remains calm, maintains perspective and responds professionally when faced with tough situations.
- **Personal Learning** - Takes personal responsibility for the continuous learning of new knowledge, skills and experiences.
- **Dealing with Ambiguity** - Able to successfully function during times of uncertainty and changing priorities.
- **Decision-Making** - Makes timely and quality decisions based on a mixture of analysis, wisdom, experience, and judgment.
- **Interpersonal Skill** - Builds effective relationships with all people; up, down, and sideways, inside and outside of Leads.
- **Results Oriented** - Gets results and achieves goals.

LOOKING FOR SW ENGINEER



Software Engineer *Experience: At least 1 year*

Additional Requirements

- Requires 1/2 years of hands-on experience in SQL.
- Requires 1/2 years' experience in the IT industry.
- Strong experience in C#, data structures and collections.
- Experience with using ASP.NET 4 Web Form, IIS and Visual Studio.
- Experience in working with Linux and Windows environments.
- Extensive understanding of SDLC and scrum methodologies.
- Experience in .Net Core would be plus.
- Experience in database development using PL/SQL would be plus.
- Experience in configuring, and deploying applications in IIS would be plus.
- Experience in using configuration management tools like Team Foundation Version Control (TFVC), Git would be plus.
- Strong interpersonal skills
- Requires excellent oral, written, and presentation skills
- Strong **analytical and problem-solving** skills
- Ability to quickly learn new application systems and technologies
- Experience in the interpretation of functional requirements
- Conduct code reviews with the team to improve compliance with established best practices and coding standards
- Work with team members to ensure application designs are in line with best practices and are scalable, reliable, and that all designs optimize performance and usability.

WHY I AM HERE



01

To Help developing your **skills** that will enable you to develop industry standard software of high quality.

02

To foster an understanding of why these **skills** are important.

WHAT IS SOFTWARE(SW)



is a **set of instructions** (computer program) and its **associated documentations** that tells a computer **what to do** or **how to perform a task**.

ATTRIBUTES OF GOOD SOFTWARE



Product characteristic	Description
Maintainability	Software should be written in such a way so that it can <u>evolve</u> to meet the <u>changing needs of customers</u> . This is a critical attribute because <u>software change is an inevitable requirement</u> of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including <u>reliability, security and safety</u> . Dependable software <u>should not cause physical or economic damage</u> in the event of <u>system failure</u> . Malicious users should not be able to access or damage the system.
Efficiency	Software <u>should not make wasteful use of system resources</u> such as <u>memory and processor cycles</u> . Efficiency therefore includes <u>responsiveness, processing time, memory utilisation, etc.</u>
Acceptability	<u>Software must be acceptable</u> to the type of users for which it is designed. This means that it must be <u>understandable, usable and compatible</u> with other systems that they use.

INDUSTRY STANDARD SW



An industry standard SW must be:

Operational

- Budget
- Efficiency
- Usability
- Dependability
- Correctness
- Functionality
- Security

Transitional

- Interoperability
- Reusability
- Portability
- Adaptability

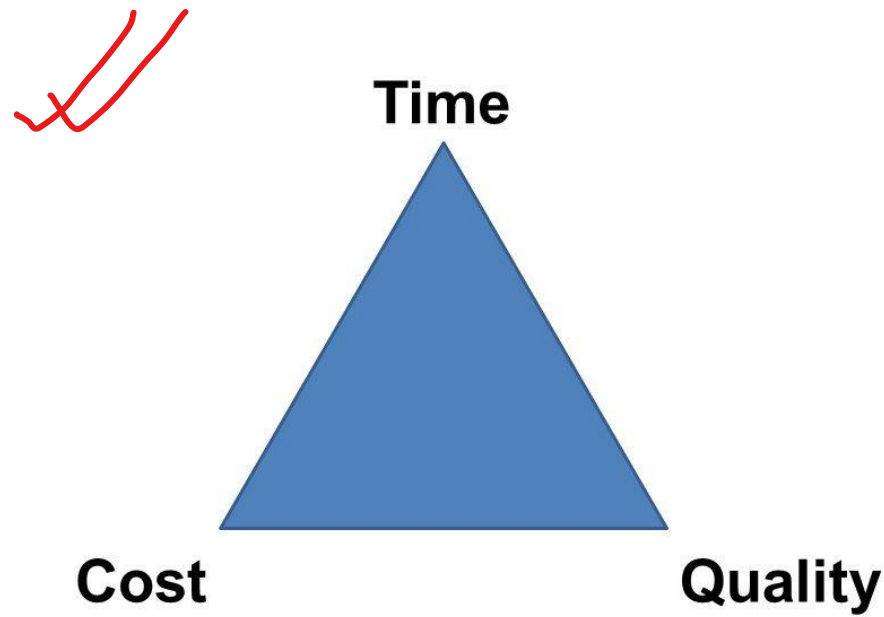
Easy to Maintain

- Flexibility
- Maintainability
- Modularity
- Scalability

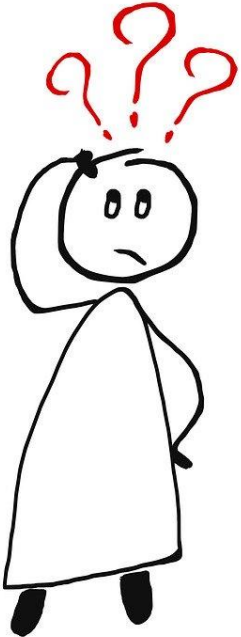
INDUSTRY STANDARD SW



- ✓ quality, cost, and schedule are the main forces that drive an industry standard software project.
- ✓ should be produced at reasonable cost, in a reasonable time, and should be of good quality.



WHAT IS SW ENGINEERING



Software engineering is defined as a discipline whose aim is the **production of quality software**, **delivered on time** and **within the budget** and that **satisfies its requirement**.

IEEE, in its standard 610.12-1990, defines software engineering as the application of a systematic, disciplined, which is a computable approach for the **development**, **operation**, and **maintenance** of software.

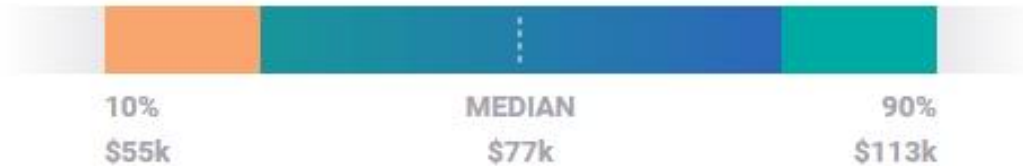
SW DEVELOPER(PROGRAMMER) VS SW ENGINEER



Average Software Developer Salary

Based on 16,538 salary profiles (last updated Jul 26'2023)

\$76,963 / year ▼
Avg. Base Salary (USD)

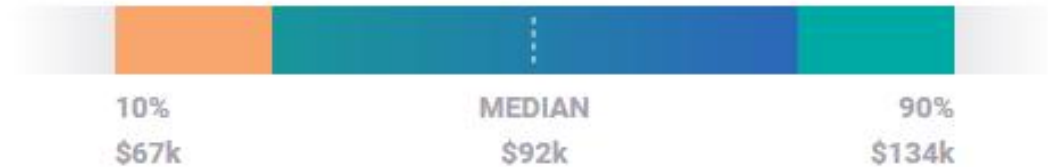


The average salary for a Software Developer is \$76,963 in 2023

Average Software Engineer Salary

Based on 48,807 salary profiles (last updated Jul 26'2023)

\$91,818 / year ▼
Avg. Base Salary (USD)



The average salary for a Software Engineer is \$91,818 in 2023

KEY DIFFERENCE



- ✓ [Software Engineer applies the principles of software engineering for designing, development, maintenance, testing, and evaluation of computer software whereas Software Developer builds software which runs across various types of computer.]
- ✓ [Software Engineer works with other components of the hardware system whereas Software Developers write a complete program.]
- ✓ [Software Engineer creates the tools to develop software] while [Software Developers use readymade tools to build apps.]
- ✓ [Software Engineer tends to solve issues on a much larger scale] whereas Software Developers tend to do everything that engineers do but on a limited scale.]



What is Industry demand?

SW Developer or SW Engineer

CHALLENGES OF SOFTWARE ENGINEERING



- ✓ In safety-critical areas such as space, aviation, nuclear power plants, etc. the cost of software failure can be massive because lives are at risk.
- ✓ Increased market demands for fast turnaround time.
- ✓ Dealing with the increased complexity of software and need for new applications.

"The complexity of software is an essential property, not an accidental one"

SOFTWARE ENGINEERING ETHICS



ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices.

The Ethical Principles are:

1. **PUBLIC** - Software engineers shall act **consistently** with the **public interest**.
2. **CLIENT AND EMPLOYER** - Software engineers shall act in a manner that is in the **best interests of their client and employer** consistent with the public interest.
3. **PRODUCT** - Software engineers shall ensure that their **products** and related modifications meet the **highest professional standards** possible.
4. **JUDGMENT** - Software engineers shall maintain **integrity and independence** in their professional judgment.
5. **MANAGEMENT** - Software engineering managers and leaders shall subscribe to and **promote an ethical approach** to the **management of software development** and maintenance.
6. **PROFESSION** - Software engineers shall advance the **integrity and reputation** of the profession consistent with the public interest.
7. **COLLEAGUES** - Software engineers shall be **fair to and supportive** of their colleagues.
8. **SELF** - Software engineers shall participate in **lifelong learning** regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

<https://ethics.acm.org/code-of-ethics/software-engineering-code/>

SW Process

SW PROCESS



- ✓ software process is a set of activities that leads to the production of a software product.
- ✓ There is no ideal process, and many organizations have developed their own approach to software development.
- ✓ For some systems, such as critical systems, a very structured development process is required. For business systems, with rapidly changing requirements, a flexible, agile process is likely to be more effective.
- ✓ fundamental activities that are common to all software processes:
 1. **Software specification** The functionality of the software and constraints on its operation must be defined.
 2. **Software design and implementation** The software to meet the specification must be produced.
 3. **Software validation** The software must be validated to ensure that it does what the customer wants.
 4. **Software evolution** - The software must evolve to meet changing customer needs.

SW Size Estimation

SW SIZE ESTIMATION



- ✓ Software size is an important metric to be used for various purposes during SE dev projects
- ✓ Difficult to Measure
- ✓ Various approaches are used to measure SW size
 - *LINE OF CODE (LOC)*
 - *TOKEN COUNT (HALSTEAD PRODUCT METRICS)*
 - *FUNCTION POINT ANALYSIS (FPA)*

SW SIZE ESTIMATION



LINE OF CODE (LOC)

- ✓ This metric is based on the number of lines of code present in the program.
- ✓ The lines of code are counted to measure the size of a program.
- ✓ The comments and blank lines are ignored during this measurement.
- ✓ The LOC metric is often presented on thousands of lines of code (KLOC).

SW SIZE ESTIMATION



LINE OF CODE (LOC)

- ✓ This metric is based on the number of lines of code present in the program.
- ✓ The lines of code are counted to measure the size of a program.
- ✓ The comments and blank lines are ignored during this measurement.
- ✓ The LOC metric is often presented on thousands of lines of code (KLOC).

SW SIZE ESTIMATION



TOKEN COUNT (HALSTEAD PRODUCT METRICS)

- ✓ LOC is not consistent, because all lines of code are not at the same level..
- ✓ Some lines are more difficult to code than others.
- ✓ **Halstead** stated that any software program could be measured by counting the number of operators and operands.
- ✓ From these set of operators and operands, he defined several formulae to calculate the **vocabulary**, the **length**, and the **volume** of the software program.

SW SIZE ESTIMATION



TOKEN COUNT (HALSTEAD PRODUCT METRICS)

✓ Halstead Metric

Program Vocabulary

- ✓ It is the number of **unique operators** plus the number of **unique operands** as given below:

$$n = n1 + n2$$

where n = program vocabulary

$n1$ = number of unique operators

$n2$ = number of unique operands

SW SIZE ESTIMATION



TOKEN COUNT (HALSTEAD PRODUCT METRICS)

✓ Halstead Metric

Program Length

- ✓ It is the **total usage of all** the operators and operands appearing in the implementation.

$$N = N1 + N2$$

where N = program length

N1 = all operators appearing in the implementation

N2 = all operands appearing in the implementation

SW SIZE ESTIMATION



TOKEN COUNT (HALSTEAD PRODUCT METRICS)

✓ Few Halstead Metric

Program Volume

- ✓ The **volume** refers to the **size of the program** and it is defined as the program length times the logarithmic base 2 of the program vocabulary.

$$V = N \log_2 n$$

where V = program volume

N = program length

n = program vocabulary

SW SIZE ESTIMATION



FUNCTION POINT ANALYSIS (FPA)

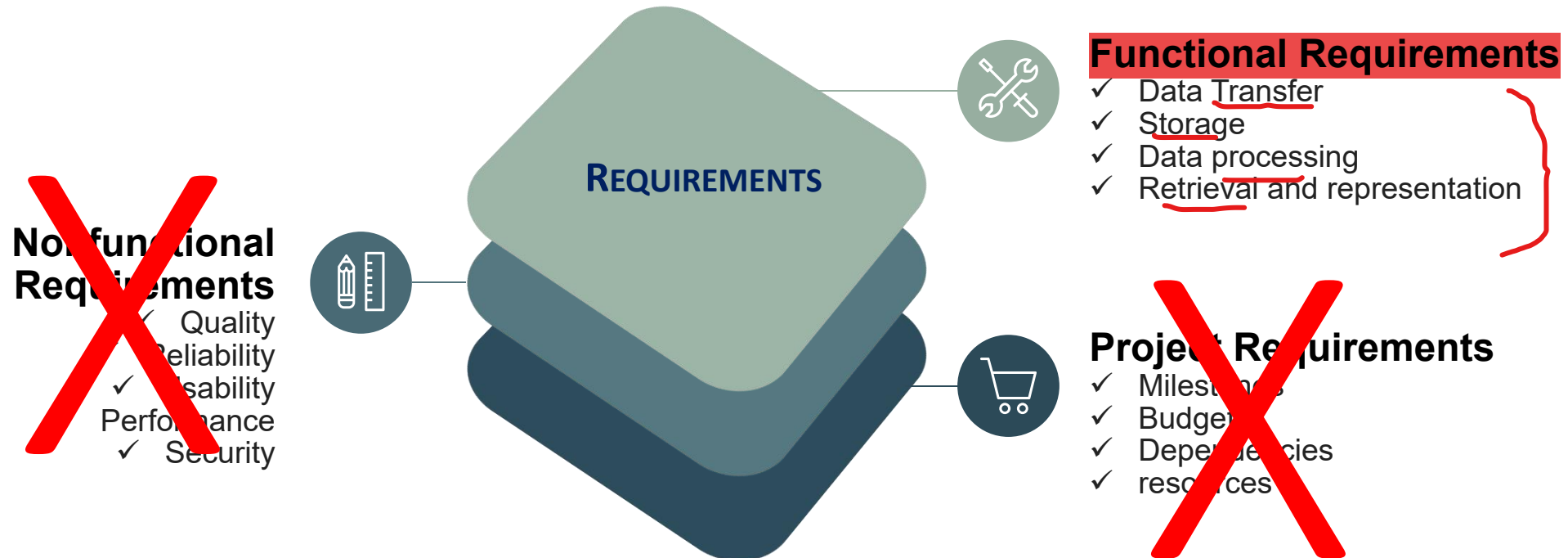
- ✓ It is based on the idea that the software size should be measured according to the functionalities specified by the user.
- ✓ Therefore, FPA is a standardized methodology for measuring various functions of a software from the user's point of view.
- ✓ The size of an application is measured in **function points**.

SW SIZE ESTIMATION



FUNCTION POINT ANALYSIS (FPA)

✓ What do Function Point measure?



SW SIZE ESTIMATION



FUNCTION POINT ANALYSIS (FPA)

Process used to calculate the function points

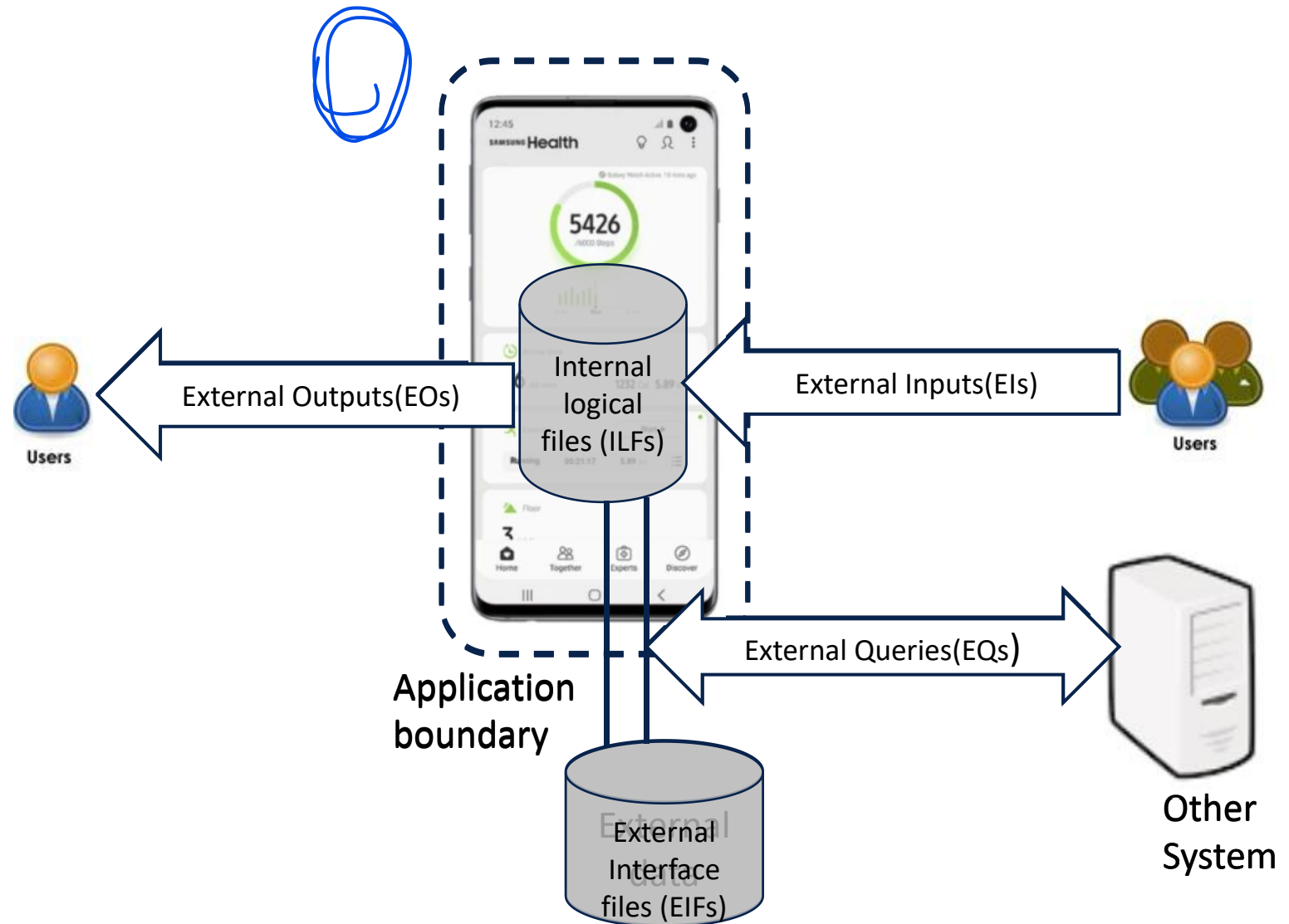
1. Determine the **type of project** for which the function point count is to be calculated. For example, development project (a new project) or enhancement project.
2. Identify the **counting scope** and the **application boundary**.
3. Identify **data functions** (internal logical file and external interface files) and their **complexity**.
4. Identify **transactional functions** (~~external inputs~~, ~~external outputs~~, and external queries) and their **complexity**.
5. Determine the **unadjusted function point count (UFP)**.
6. Determine the **value adjustment factor (VAF)**, which is based on **14 general system characteristics (GSCs)**.
7. Calculate the **adjusted function point count (AFP)**.

SW SIZE ESTIMATION



FUNCTION POINT ANALYSIS (FPA)

- ✓ The first step in calculating FP is to identify the counting boundary.
- ✓ Counting boundary: The border between the application or project being measured and external applications or the user domain.
- ✓ A boundary establishes which functions are included in the function point count



SW SIZE ESTIMATION



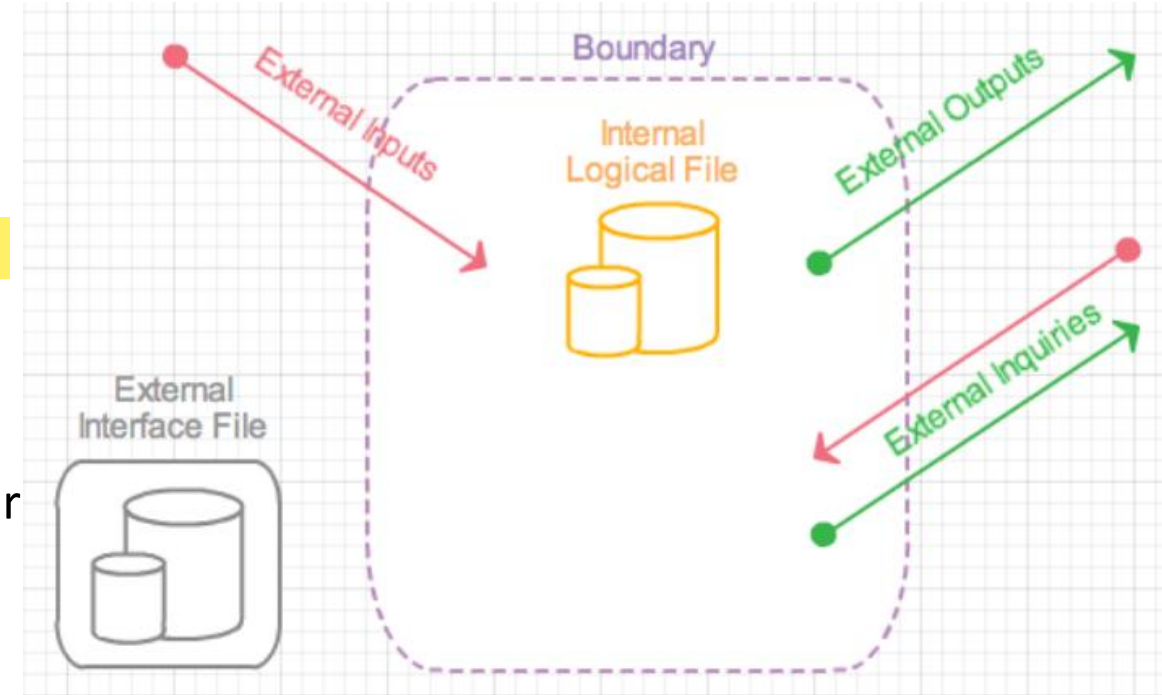
Size Metric: *FUNCTION POINT ANALYSIS (FPA)*

✓ **Two** data files/function types:

- **Internal Logical Files (ILF):** This is the set of data **present within the system**. The majority of the data will be interrelated and are captured via the inputs received from the external sources.
- **External Interface Files (EIF):** represent the data that your **application will use/reference**, but **data that is not maintained by your application**.

✓ **Three Transaction types:**

- **External Inputs (EI):** these are **end-user actions** such as putting in a login or executing a mouse click.
- **External Outputs (EO):** the system provides the **end-user output or interface** such as a GUI display or items in a report.
- **External Inquiries (EQ):** this function is **initiated by the end-user**. For example, the end-user wishes to submit a query to a database or requests on-line help. In any case the developer provides a means for the end-user to "search" for answers.



SW SIZE ESTIMATION



Size Metric: *FUNCTION POINT ANALYSIS (FPA)*

- ✓ Based on the data function complexity weight is assigned to each data function

Measurement Parameter	Low	Average	High
1. external inputs (EI)	3	4	6
2. external outputs (EO)	4	5	7
3. external inquiries (EQ)	3	4	6
4. internal files (ILF)	7	10	15
5. external interfaces (EIF)	5	7	10

- ✓ Count the number of all five components. The sum of each count is multiplied by an appropriate weight using Table above
- ✓ Add all the five results calculated in the previous step. This is the final **UFP**.

Function Type	Functional Complexity	Complexity Totals	Function Type Totals
ILFs	Low	X 7 =	
	Average	X 10 =	
	High	X 15 =	
EIFs	Low	X 5 =	
	Average	X 7 =	
	High	X 10 =	
EIs	Low	X 3 =	
	Average	X 4 =	
	High	X 6 =	
EOs	Low	X 4 =	
	Average	X 5 =	
	High	X 7 =	
EQs	Low	X 3 =	
	Average	X 4 =	
	High	X 6 =	
Total Unadjusted Function Point Count			

SW SIZE ESTIMATION



Size Metric: *FUNCTION POINT ANALYSIS (FPA)*

✓ Calculating **Adjusted Function Point:**

- ✓ A **value adjustment factor (VAF)** is used as a multiplier of the unadjusted function point (**UFP**) count in order to calculate the adjusted function point (**AFP**) count of an application.
- ✓ To calculate the VAF, we evaluate the 14 GSCs (General system Characteristics) on a scale of 0–5 to determine the degree of influence (DI) for each GSC description.
- ✓ Add the DIs for all 14 GSCs to produce the **total degree of influence (TDI)**.
- ✓ Use the TDI in the following equation to compute VAF.
- ✓ The final adjusted function point is calculated as,

0 = No Influence
1 = Incidental
2 = Moderate
3 = Average
4 = Significant
5 = Essential

Factor	Meaning
F1	Data communications
F2	Performance
F3	Transaction rate
F4	End user efficiency
F5	Complex processing
F6	Installation ease
F7	Multiple sites
F8	Distributed data processing
F9	Heavily used configuration
F10	Online data entry
F11	Online update
F12	Reusability
F13	Operational ease
F14	Facilitate change

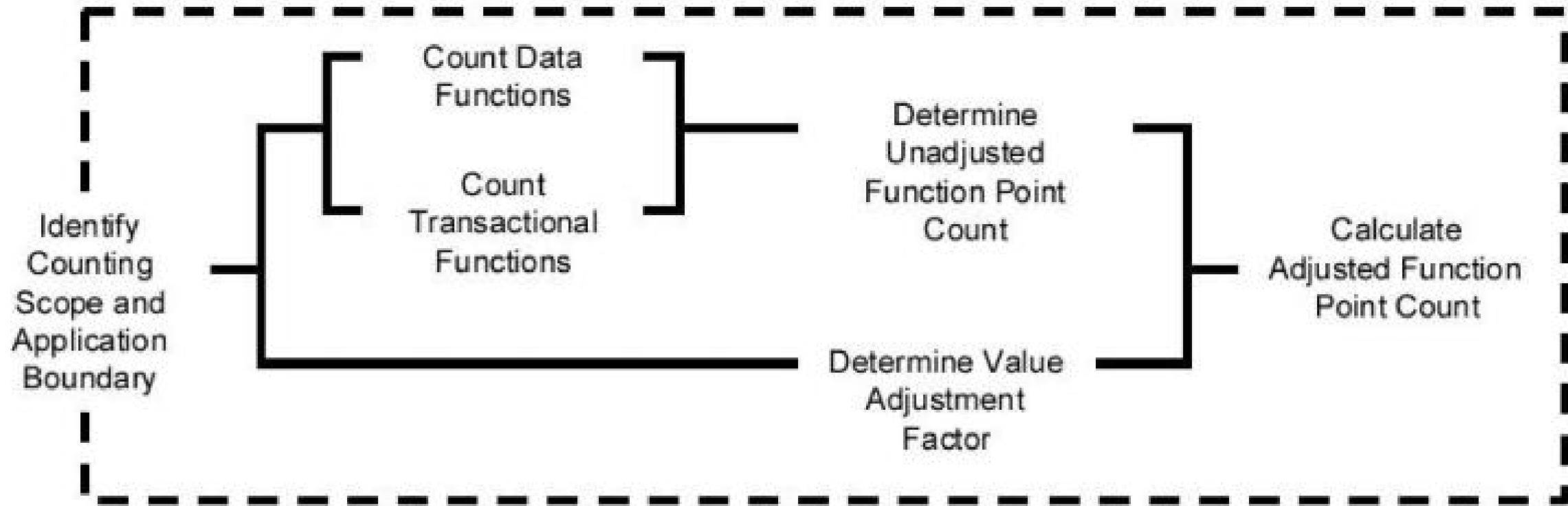
$$VAF = (TDI \times 0.01) + 0.065$$

$$AFP = UFP \times VAF$$

SW SIZE ESTIMATION



✓ FUNCTION POINT ANALYSIS (FPA)



SW SIZE ESTIMATION



FUNCTION POINT ANALYSIS (FPA)

✓ Example:

Consider a project with the following parameters: EI = 50, EO = 40, EQ = 35, ILF = 06, and ELF = 04. Assume all weighing factors are **average**. In addition, the system requires **Significant** performance, **average** end-user efficiency, **moderate** distributed data processing, and **Significant** data communication. Other GSCs are **incidental**. Compute the function points using FPA.

SW Development Life Cycle (SDLC)

SW DEVELOPMENT LIFE CYCLE (SDLC)



- ✓ is a systematic process for building software that ensures the **quality** and **correctness** of the software built.
- ✓ SDLC aims to produce **high-quality** software that meets **customer expectations**.

✓ WHY SDLC



- ✓ It offers a basis for project planning, scheduling, and estimating
- ✓ Provides a framework for a standard set of activities and deliverables
- ✓ It is a mechanism for project tracking and control
- ✓ Increases visibility of project planning to all involved stakeholders of the development process
- ✓ Increased and enhance development speed
- ✓ Improved client relations
- ✓ Helps you to decrease project risk and project management plan overhead

SDLC PHASES



Phase 1: Requirement gathering and analysis

Phase 2: Feasibility

Phase 3: Design

Phase 4: Coding

Phase 5: Testing

Phase 6: Installation/Deployment

Phase 7: Maintenance

REQUIREMENT GATHERING AND ANALYSIS



- ✓ is one of the most critical phases of SDLC.
- ✓ the basis of the whole software development process.
- ✓ All the business requirements are gathered from the client in this phase.
- ✓ Requirements are analyzed to prepare SW specification, A well defined document known as Software Requirement Specification (SRS)

is divided into two phases where **requirement gathering** is related to identify client's projects technical requisites and the **Analysis** is to evaluate whether the accumulated information and facts are valuable enough to proceed with the project plan.

REQUIREMENT GATHERING AND ANALYSIS



- ✓ Why is the requirement gathering important?
 - is all about eliciting project requisites from client end so that they can get a full-fledged product embedded with all the specifications.
 - it helps all the project associates communicate effortlessly with each other over defined clients needs.

- ✓ Who is responsible for requirements gathering?
 - Business Analysts are the professionals who efficiently carry out software requirement gathering by breaking down the critical technical specifications into effective documentation and user stories.

REQUIREMENT GATHERING AND ANALYSIS



✓ What are the tools and techniques of requirement gathering?

the responsible person observes the team in working environment and gets ideas about the software and subsequently document the observation.

Observation



Interview



Survey

Questionnaire delivered to the user as well as the stakeholder for answers.

subject matter experts are the responsible people to conduct brainstorming sessions. They discuss and find out solutions to complex issues.

Brainstorming



REQUIREMENT
GATHERING



Joint Application
Method

all the stakeholders like developers, end users, business analysts and software engineers come together and attend workshops for working on a system in greater detail.

the idea is to collect information from representatives of clients and users to understand the software idea clearly.



Focus Group



Interface Analysis

Interface analysis is a specialized technique in which specific requirements related to application development are determined and their interaction with other software components is measured.

Use case diagram is a technique that shows how people interact with software. It shows what a system does.

Used Case Diagram



Prototyping

building a model of software which helps in uncovering and capturing software requirements from client.

REQUIREMENT GATHERING AND ANALYSIS



Challenges in Requirements Gathering

*Lack of Clarity
in Defining
Criteria for
Success*

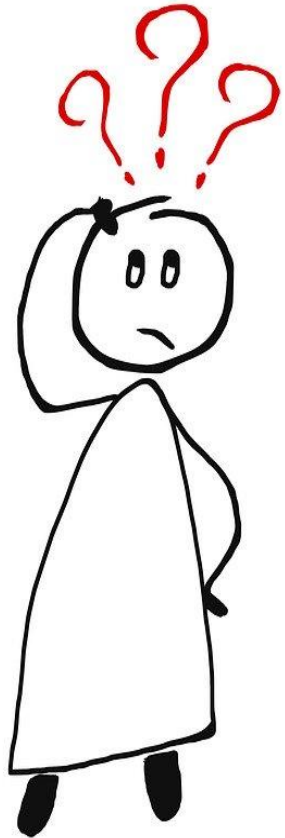
*Clients Change
their Minds
Often*

*Lack of or
Over
Communication
by Clients*

*Clients get
stuck over
certain
Techniques/Sol
utions*

*Stakeholders
can have
Conflicting
Priorities*

REQUIREMENT GATHERING AND ANALYSIS

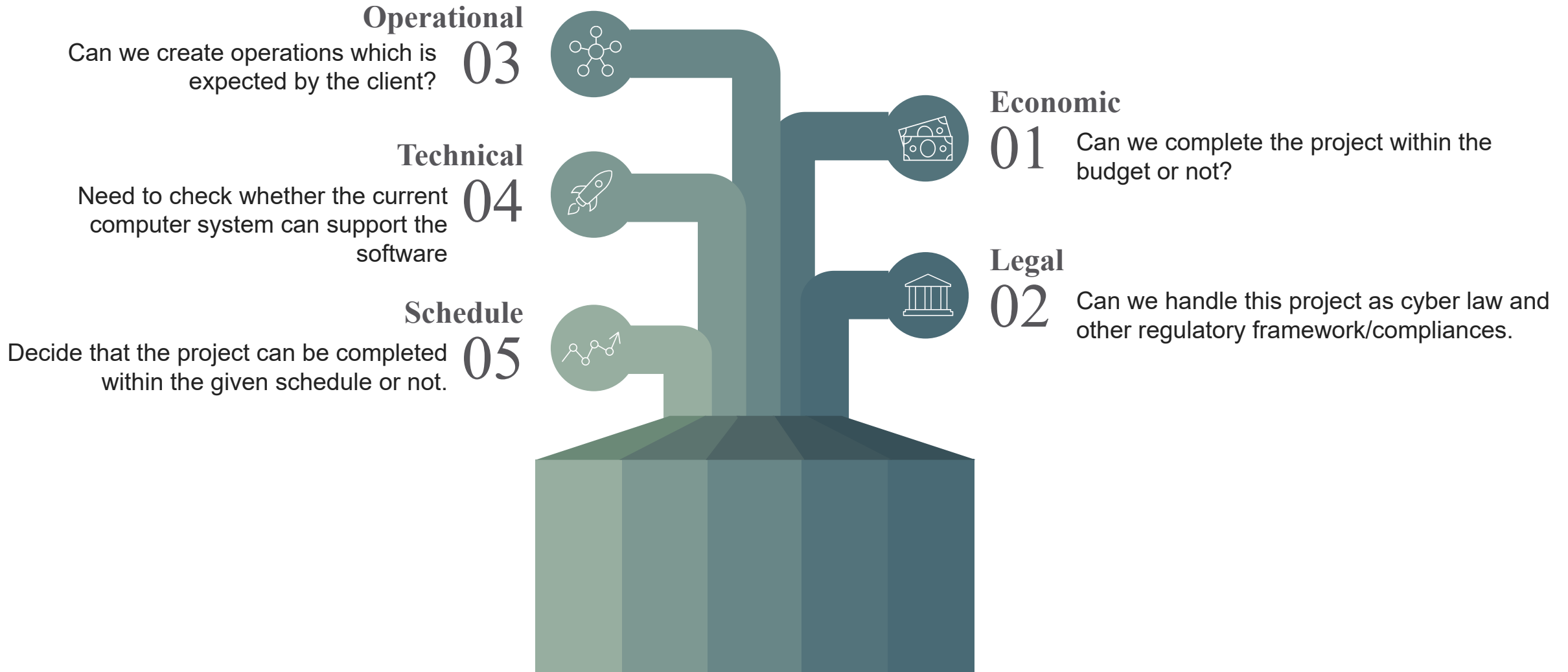


You are business analyst of XYZ Ltd. ABC bank wants to develop a mobile application of their banking products for their end users. Your company appointed you to collect the requirements for the mobile application from ABC Bank. **What technique will you use to collect the requirement and what are the answers you need to get from your stakeholders??**

FEASIBILITY



Five types of feasibility check we need to perform



- ✓ Design documents are prepared as per the SRS.
- ✓ Helps to define overall system architecture.

There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- ✓ Brief description and name of each module
- ✓ An outline about the functionality of every module
- ✓ Interface relationship and dependencies between modules
- ✓ Database tables identified along with their key elements
- ✓ Complete architecture diagrams along with technology details

Low-Level Design(LLD) / Detail Level Design (DLD)

- ✓ Functional logic of the modules
- ✓ Database tables, which include type and size
- ✓ Complete detail of the interface
- ✓ Addresses all types of dependency issues
- ✓ Listing of error messages
- ✓ Complete input and outputs for every module

Why software design is important?



Modularity

When you have a modular software, it's easy to move or even remove things if needed and even share the design work between multiple developers per software component.



Performance

design should tell how your system works internally and how it uses resources such as threads, database connections and other things that might make a hit to the performance.



Usability

design works as a great starting point for new-comers. Instead of making them look through all the source files you can just point them to the software design where they find all information they'll need to get started.



Maintainability

With good software design it's easier to maintain the software. You can spot straight away from the design how much a bug fix or introduction of a new feature will change the existing code base.



Portability

By including dependencies to other software modules, such as 3rd party libraries, in to your design makes the software much easier to port into another environment.



Trackability

Good design tracks the requirements and proves on the design level that all that is required from a piece of software is actually there..

- ✓ developers start build the entire system by writing code using the chosen programming language.
- ✓ tasks are divided into units or modules and assigned to the various developers.
- ✓ is the longest phase of the Software Development Life Cycle process
- ✓ developer needs to follow certain predefined coding guidelines
- ✓ also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Things which we need to take care in coding phase:

1. Version control
2. Before actual coding, you must spend sometime to select dev tool which will be suitable for debugging, coding, modification.
3. before actual writing code, some standard should be defined, as multiple developers going to use the same file for coding
4. during development, developer should write appropriate comments so that others will come to know the logic behind the code
5. there should be a regular review meeting need to conduct in this stage to identify the prospective defects in an early stage.

TESTING



- ✓ Focus on investigation and discovery
- ✓ defines how the SW will be tested and who will perform
- ✓ find whether the developed code work according to the customer requirement.
- ✓ QA and testing team may find some bugs/defects which they communicate to developers.
- ✓ development team fixes the bug and send back to QA for a re-test.
- ✓ process continues until the software is bug-free, stable, and working according to the business needs of that system.

INSTALLATION/DEPLOYMENT



- ✓ Deployment is the process of promoting a fully-tested and approved application
- ✓ QA manager finalized the final bug-free release items.
- ✓ Deployment guide is provided to the operation team for hassle free deployment
- ✓ Deployment could require different promotion processes
 - ✓ For web applications, it would require promoting the application to a server, testing, redirecting DNS
 - ✓ For standalone applications, it would require producing media or downloadable packages
 - ✓ For software as a service, there are licensing issues that need to be determined
- ✓ Ensure data migration to the live server has occurred completely

MAINTENANCE



Major Activities after SW deployment/while SW in use by the end users



The main focus of this phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.



Why do we need a SW upgrade?